

The Knapsack Problem

- **Input:** n item types, with:
 - Sizes $s_1 \dots s_n$ (all integer).
 - Values $v_1 \dots v_n$.And also a capacity C knapsack.
- **Goal:** Find a maximum-value collection of items that fits in the knapsack.
- This will give us our first formulation involving a “2D” table of subproblems...
- (Note: This problem is actually NP-hard; our $O(nC)$ running time doesn't count as “polynomial time” due to direct dependence on C)

Edit Distance / Dynamic Time Warping

- A common way to measure (dis-)similarity between A and B is via the minimum cost of transforming A into B.
- Several prominent problems fit this theme:
 - **Edit Distance:** given costs for inserting, deleting, or modifying characters, what is the minimum cost to transform string $A[1\dots m]$ into string $B[1\dots n]$.
 - **Dynamic Time Warping:** similar, but where we are aligning time series data $f[1\dots m]$ and $g[1\dots n]$, with the cost of aligning value x with value y typically being $|x - y|$.

Binary Search with Asymmetric Guessing Penalties

- Searching a sorted array $A[1 \dots N]$ for a target value v .
- Guessing too low costs \$1
- Guessing too high costs \$2
- What amount of money do you need in your pocket to be able to guarantee you can find v , irrespective of where it is in the array?

(Note, answer is at most $2 \log N$, the worst case of standard binary search)